Smriti — Technical Defense Brief

For the person on stage. Written against the code at commit `fa3e275`. Every claim carries a `file:line`. Fifteen minutes, hostile room.

PART 1 — THE SYSTEM IN ONE PAGE

What it is. An offline-first cognitive-**engagement** and memory-assistance PWA for elderly people living with dementia in North-East India. It speaks the day's reminders aloud, runs five practice activities that adapt to how the activities are going, shows named photographs of the patient's people, and gives a caregiver a plain record of what happened.

What it refuses to be. Not diagnostic. Not a medical device. Not treatment. It makes no claim to detect, diagnose, measure or improve cognition. Every number it reports describes activity performance inside the app — a rule written into the two modules most likely to break it (`js/core/analytics.js:5-6`, `js/core/model.js:12-13`) and printed on all three caregiver screens (`content/en.json:640, 696, 743`).

The five constraints that shaped every decision.

1. **Offline-first.** Airplane-mode functional after one load. This is why there is no charting library, no CDN, no cloud model, no analytics.
2. **No build step.** Ships exactly what is in the repo. This is why `APP_SHELL` is a hand-written list (`sw.js:3-80`) and why the module graph is explicit.
3. **No framework.** 3,654 lines of JavaScript, zero dependencies, so

it starts fast on an old browser and can be read end to end in an afternoon.
4. **No login.** Entry is a role toggle, not a sign-in. All data is one device's.
5. **Low-end Android (~1 GB).** This is why reads are synchronous, why photos are downscaled before storage, and why speech never blocks a screen.

Module map. Arrows point from dependent to dependency.

File	Owns	Depends on
`index.html`	17 lines: one `<div id="app">`, three stylesheets, one module	—
`js/app.js`	boot order; injects the content pack into every route	content, store, demo-data, voice, router, all screens
`js/core/store.js`	**all** persistence: 5 collections, IndexedDB + localStorage mirror, sync read cache	adaptive, model
`js/core/router.js`	hash routes; tears down speech + mic on every route change	voice, listen
`js/core/content.js`	loads `content/<lang>.json`, falls back to `en`	—
`js/core/voice.js`	recorded clips first, live `speechSynthesis` second; cancellation correctness	—
`js/core/listen.js`	optional STT + the conservative spoken-answer matcher	—
`js/core/adaptive.js`	the rules engine (3 right → up, 2 wrong → down). **Pure.**	—

`js/core/model.js`	the on-device learner. Bounded by adaptive. **Pure.**	adaptive
`js/core/analytics.js`	derived views + alerts. **Pure, stores nothing.**	—
`js/core/chart.js`	hand-built inline SVG bar chart + sparkline	dom
`js/core/dom.js`	`el()` / `svgEl()` — 32 lines, the whole view layer	—
`js/core/format.js`	time and "when" phrasing, words from the pack	—
`js/core/demo-data.js`	the first-run sample payload	adaptive, model
`js/screens/entry.js`	the only screen both worlds share	dom
`js/screens/patient/*`	home, today, memories, exercises, reminder-card	dom, shared, voice, listen, store
`js/screens/caregiver/*`	dashboard, overview, reminders, engagement, memories-settings, alerts	dom, shared, format, chart, analytics, model, store, voice
`content/en.json`, `as.json`	**every** user-visible string, image path and TTS language chain — 274 keys each, identical shape	—
`sw.js`	offline cache: 79 shell entries + English audio read from its index at install	—
`tools/*`	voice build, model harness, four browser self-tests. Never shipped.	—

Three structural facts that answer "how does it work" on their own:

`adaptive.js`, `model.js` and `analytics.js` are pure functions with no I/O; `model.js` imports `adaptive.js` and never the reverse, so the learner is bounded by the rules and not the other way round; and only `store.js` writes to disk.

PART 2 — THE TWELVE THINGS THAT MAKE IT NON-TRIVIAL

---

1. Pre-rendered neural voice, with live synthesis as the fallback

Decision. Every fixed patient-facing line is rendered once at build time with a neural voice and shipped as an MP3; only lines assembled at runtime fall through to the device engine.

Naive alternative. Call `speechSynthesis.speak()` for everything. On a low-end Android that is a robot reading a form, it sounds different on every phone, and on the one phone that matters — the demo device — it may have no voice installed at all, in which case the app is silent.

Evidence.

```
  pending = setTimeout(() => {
    pending = null;
    if (gen !== generation) return;
    if (file && playClip(file, line, onEnd, gen)) return;
    speakLive(line, onEnd, gen);
  }, 60);
```

— `js/core/voice.js:136-141`

Result. 191 English and 189 Assamese lines are recordings. The device engine is needed for exactly 8 templated lines. A phone with no TTS

engine still speaks the intro, all feedback, the praise, and the Exercise 2 and 5 prompts.

---

2. Synchronous reads over an async database

Decision. `initStore()` hydrates every collection into memory once at boot; every screen then reads synchronously. Writes update memory, write the localStorage mirror immediately, and queue the IndexedDB write.

Naive alternative. `await` IndexedDB inside each render function. Every patient screen then has a frame where it is blank — on a 1 GB phone that reads as a stutter or a crash, and it makes every render function async for no benefit.

Evidence.

```
// Reads are synchronous against an in-memory cache hydra
// so screens can render without awaiting. Writes update
// and persist in the background through a serialised que
```

— `js/core/store.js:7-9`

Result. Four awaits in the entire app, all in `bootstrap()` (`js/app.js:22-41`). `tools/store-test.html` proves it is real persistence, not a cache: it plants a marker on one page load and asserts it survived on the next.

---

3. A wrong answer never skips the round

Decision. In four of five exercises a wrong tap is marked, gently narrated, cleared, and the round reopens — input is held during the pause. In the fifth, every option locks on the first tap so later taps do nothing at all.

Naive alternative. Advance on any answer. A patient tapping

repeatedly — which is exactly what someone uncertain does — skips the whole session in seconds and is told nothing.

Evidence.

```
        // Wrong tap: no reveal, no skip — just a nudge and
        busy = true;
        const missed = buttons.filter((b) => b.dataset.opti
        if (missed) missed.classList.add('is-passed-over');
        sayThen(c.feedback.retry, feedback, 'gentle', () =>
          if (missed) missed.classList.remove('is-passed-ov
          clearFeedback(feedback);
          busy = false;
          speak(promptSpeech);
        }, RETRY_MS);
```

— `js/screens/patient/exercises.js:549-558`

Result. Asserted by the driven-session test: *"repeated taps during the pause do not skip the round"* and *"every option locks after the first tap"* (`tools/demo.html:217-222`).

---

4. The learner may move one level, and only one, from where the rules landed

Decision. The rules engine decides first. The learner may then pick `base - 1` or `base + 1`, only if that is at least 0.03 closer to the 75 % target, with ties falling to the easier level — and it does nothing at all for its first six rounds.

Naive alternative. Let the model choose freely. One lucky streak and a patient is at the hardest level; one bad patch and the level oscillates every round. Neither is explainable to a clinician.

Evidence.

```
// Picks the level to play next. The rules engine has alr
```

```
// wants to be; the model may move one step either side o
// it is clearly better. Ties fall to the easier level.
```

— `js/core/model.js:117-119`

Result. Brute-forced: 1,200 proposals against random 30-round histories, **zero** landing more than one level from the rules' choice (`tools/model-test.html:105-117`). And the guardrail costs nothing — see #12.

---

5. Speech matching that refuses to guess

Decision. A spoken answer resolves only if the best-matching option clears **0.7** absolute and leads the runner-up by **0.12**. Otherwise nothing happens and the patient is told plainly it was not heard — never that they were wrong.

Naive alternative. Take the best match. A garbled transcript then invents a mistake the patient never made: it trains the learner on a lie, and it tells someone with memory loss they were wrong when they were not.

Evidence.

```
    Deliberately conservative. Resolving the WRONG option
    transcript was garbled would invent a mistake the pati
    and a false wrong answer is far worse here than hearin
    all: it trains the learner on a lie and it tells someo
    loss they were wrong when they were not. So a match mu
    strong and unambiguous, or nothing happens.
```

— `js/core/listen.js:114-119`

Result. 18 cases in `tools/listen-test.html`, verified: every intended match resolves (including "I think that's the gamosa" and "jappi"), and every one of the eight cases that must resolve to **nothing** does — noise, a filler, "I do not know", a question, a two-letter fragment, and two equally confident alternatives.

---

6. Only English audio is precached

Decision. The install precaches the shell plus the English clips — **261 KB + 3.26 MB = 3.51 MB**. Assamese clips are cached opportunistically the first time each one plays.

Naive alternative. Precache both languages and double the install to ~6.8 MB on a rural connection, for a language most devices will never select.

Evidence.

```
// Only English is precached up front. The Assamese set i
// and a device that never switches language should not p
// connection — the fetch handler caches those clips the
// and until then that language simply speaks live.
```

— `sw.js:86-89`, with the write-back at `sw.js:143-146`

Result. Half the install size, and switching language is a one-off cost rather than a permanent dependency on being online. The honest cost: a device that switches to Assamese while offline speaks that language live until it has been online once.

---

7. Assamese falls back to Bengali — in both layers

Decision. The recording build renders Assamese with a Bengali neural voice, and the live fallback chain is `as-IN → as → bn-IN → bn`.

Naive alternative. Fall back to English. A Latin-script voice reading Bengali-Assamese script produces sounds that are not words.

Evidence.

```
# Assamese neural voices do not exist. Assamese is writte
# Bengali-Assamese script, so a Bengali voice pronounces
```

```
# Latin-script voice — the same reasoning voice.js alread
# fallback chain.
VOICES = {
    'en': 'en-IN-NeerjaNeural',
    'as': 'bn-IN-TanishaaNeural',
}
```

— `tools/build-voice.py:41-48`; chain at `content/as.json:8-13`; matcher at `js/core/voice.js:87-91`

Result. The chain prefers a real Assamese voice where one exists and degrades to the nearest script-correct one where it does not — and Caregiver Settings reports which voice the device actually found, so nobody has to guess (`js/screens/caregiver/memories-settings.js:166-198`).

---

8. Language is data, not code

Decision. Every user-visible string, every image path and the ordered TTS language chain live in `content/<code>.json`. The router hands the loaded pack to every render function through one closure.

Naive alternative. A `t()` lookup sprinkled through the screens, or worse, strings in the JavaScript. Adding a language then means touching every file, and "the pack is complete" becomes unverifiable.

Evidence.

```
function withContent(renderFn, content) {
  return (container, params, navigate) => renderFn(contai
}
```

— `js/app.js:18-20`; loader at `js/core/content.js:8-17`

Result. The two shipped packs are structurally identical: **274 keys each, with no key present in one and missing from the other** (verified by walking both trees). A third language is a file plus one entry in `app.languages` — no application code changes. Be precise on stage:

two things *outside* the app do change, a line in `tools/build-voice.py` if you want recordings and an entry in `sw.js` `APP_SHELL` so it precaches.

---

9. The reminder watcher

Decision. A 30-second poll, started once at boot, that surfaces a due reminder wherever the patient is — except inside an exercise.

Naive alternative. Check only when Patient Home renders. A reminder then comes due while the patient is sitting looking at the screen and nothing happens.

Evidence.

```
// Routes where a reminder may interrupt. An exercise in
// alone: the card would break the round, and the patient
// It surfaces as soon as they come back to Home.
const INTERRUPTIBLE = ['patient-home', 'today', 'memories
```

— `js/screens/patient/reminder-card.js:16-19`; watcher at `:60-74`

Result. Reminders arrive on time while the app is open, and never break a round in progress. Two cards can never stack (`js/screens/patient/reminder-card.js:62-63`). Stated limit, in the source and the README: it cannot fire while the app is closed.

---

10. Token-based teardown of a whole session

Decision. An exercises session captures an incrementing token; every timer checks it before firing, and leaving the screen bumps it, clears the timers, stops speech and closes the microphone.

Naive alternative. Bare `setTimeout`. The patient taps Home, and four seconds later a timer from the abandoned session advances a screen

that no longer exists — or the mic outlives the screen that opened it.

Evidence.

```
  // Called before any route change out of the session.
  function leave() {
    activeToken += 1;
    clearTimers();
    stopSpeaking();
    // The microphone must never outlive the screen that
    stopListening();
  }
```

— `js/screens/patient/exercises.js:104-111`; guard at `:77-89`

Result. The same discipline was then lifted to the router, so it covers screens that do not speak on entry at all — Patient Home and every caregiver screen (`js/core/router.js:25-31`).

---

11. A generation counter that tells "cancelled" apart from "failed"

Decision. Every `speak()` and `stopSpeaking()` bumps a counter. A line captures it when it starts and checks it before acting on any callback that arrives later.

Naive alternative. Trust the callbacks. Both engines lie in the same direction: pausing an `<audio>` whose `play()` has not resolved rejects that promise with `AbortError` — indistinguishable from a broken file — and `speechSynthesis.cancel()` fires `onend` on the utterance it just killed.

Evidence.

```
// Both engines report a line we deliberately stopped as
// to it: ... Without this counter the first is indisting
```

```
// from a broken file — so the line falls back to the liv
// itself again over the top of whatever started next — a
// the caller a line finished when it never did.
let generation = 0;
```

— `js/core/voice.js:24-32`

Result. Two concrete bugs closed: a cancelled recording re-speaking itself over the next screen (`js/core/voice.js:170-176`), and a screen advancing on a sentence the patient never heard (`js/core/voice.js:207-219`). It has its own device test — `tools/voice-test.html:113-159` — because it depends on real audio timing.

---

12. A validation harness that cannot drift from the code

Decision. `tools/model-harness.py` mirrors the learner in Python but reads `TARGET_SUCCESS`, `LEARNING_RATE`, `WEIGHT_DECAY`, `WINDOW`, `MIN_ROUNDS_TO_STEER`, `MIN_GAIN`, `INITIAL_WEIGHTS` and the four rules-engine constants **straight out of the JavaScript source** by regex. A golden 20-round trace then pins the shipped JS to the validated Python.

Naive alternative. Copy the constants into the harness. Six months later the harness proudly validates a model nobody is running.

Evidence.

```
M = read_constants('js/core/model.js', [
    'TARGET_SUCCESS', 'LEARNING_RATE', 'WEIGHT_DECAY', 'W
    'MIN_ROUNDS_TO_STEER', 'MIN_GAIN', 'INITIAL_WEIGHTS',
])
```

— `tools/model-harness.py:47-50` (and `:51-53` for the rules-engine constants); the reader itself at `:32-44`; golden trace at `:231-251`, asserted in the browser at `tools/model-test.html:88-99`

Result — the numbers to quote. 4 synthetic patients × 40 seeds × 400

rounds:

```
patient                              best   learner     ru
steady (best at level 2)            74.0%     75.6%     62
strong (best at level 3)            76.0%     79.9%     77
struggling (best at level 1)        72.0%     70.7%     56
declining over time                 60.3%     58.0%     52
```

The learner beats the rules engine alone by **12.7, 2.2, 14.5 and 5.6 points** of realised success, lands within 5 points of the best any single level could have given, changes level on only 2–8 % of rounds, and its predictions are within 0.046–0.077 of the true probability. And I re-ran `--golden` against current source: the three constants pinned in `tools/model-test.html:36-38` match exactly, so the mirror has not drifted.

PART 3 — THE INTERROGATION

Twenty questions, most dangerous first.

---

1. "You said no ML, but you shipped a trained model. Which is it?"

A. The classifier is real and calling it machine learning is correct. The rule this project set itself forbids three specific things — an LLM, a cloud dependency, and a downloaded model file — and the code violates none of them: it starts from four hard-coded numbers, trains only from this patient's own rounds on this device, and never touches the network. Where the documentation is wrong is `README.md:75-76`, which still says "there is no ML"; that line predates the learner and should read "no cloud ML, no LLM, no pretrained model." The *user-facing* copy has been correct throughout — the caregiver screen calls it "On-device learning" and explains exactly what it does.

PROOF. `js/core/model.js:1–17`; `content/en.json:704–710`; the stale line at `README.md:73–76`.

TRAP. Do not say "well, it's only logistic regression, that's not really ML." It is, the room knows it is, and hedging turns a defensible design into an apparent cover-up. Own the word, then show the guardrail.

---

2. "Where is the AI? This is if-statements."

A. There are two layers and they do different jobs. `adaptive.js` is 54 lines of if-statements — three right in a row moves up, two wrong eases back — and it is deliberately that simple, because it is the auditable floor you can explain to a caregiver in one sentence. `model.js` is a logistic-regression classifier trained on-device by SGD from that patient's own rounds; it predicts the chance of success at each level and picks the one nearest 75 %. Rules for the guarantee, a learner for the fit — and the learner is measurably better than the rules alone, by 12.7 points of realised success on the steady patient.

PROOF. `js/core/adaptive.js:26–54` and `js/core/model.js:100–149`; harness table in Part 2 #12.

TRAP. Do not inflate it — no "AI-powered", no "intelligent system". Inflation invites exactly the follow-up you cannot survive. The strongest possible framing is the true one: 100 lines of arithmetic a clinician can audit.

---

3. "Prove the adaptive part does anything."

A. Two ways. Offline: the harness runs four synthetic patients with known true per-level success rates for 400 rounds each across 40 seeds, and the learner lands within 5 points of the best any single level could have given, beats the rules engine on all four, and changes level on only 2–8 % of rounds. Live, on this phone: the demo device is seeded one correct answer short of a level rise in the first two activities, so three correct answers in Remembering moves the level in front of you

— the patient hears "let's try a little harder", and the Engagement card changes. That is staged *starting state*, not staged rules.

PROOF. `tools/model-harness.py:293-301`; `js/core/demo-data.js:116-125`; `js/screens/patient/exercises.js:181-186`.

TRAP. Do not claim the harness numbers say anything about real patients. Say "synthetic patients, so this validates the control loop, not clinical benefit" before they say it for you.

---

4. "You claim offline. Does speech recognition work in airplane mode?"

A. No. It does not, and it cannot. The browser's recogniser is cloud-backed on Android Chrome — it streams audio to a server, and there is no offline speech recognition available to a web page. So speech input is an enhancement where there is signal, never a requirement: tapping always works, nothing in the app waits on speech, and a network error retires the microphone for the session rather than leaving a dead button on screen. Everything else works with the radio off — every screen, all five activities, both content packs, every image, all 191 recorded English lines, the learner and the store are precached, 3.5 MB total.

PROOF. `js/core/listen.js:11-16` and `:77-85`; `sw.js:116-124`; `content/en.json:733`.

TRAP. Do not say "it works offline" and hope. And do not blame the browser without owning the design consequence — the sentence that wins the room is "it is the one thing that breaks offline, which is why it can never be required to advance."

---

5. "This handles vulnerable patients' data with no login and no encryption."

A. Correct, and both are deliberate. Encryption at rest needs a key; a

key needs a password, and a password in front of a person with dementia is a barrier they cannot clear — or it needs a hardware keystore a web page cannot reach. Storing a key beside the data would be a claim, not a protection. What we built instead is real: nothing leaves the device, no accounts, no server, no analytics, no CDN. The exposure is one phone's worth of data to whoever holds that phone — not a database of patients. For a deployed product the answer is a native wrapper with the OS keystore and device encryption, and that is roadmap, not demo.

PROOF. `js/core/store.js:1-5`; `README.md:98-106`; `js/screens/caregiver/memories-settings.js:250-272`.

TRAP. Do not call the long-press corner a security feature. It is a UX gate and the code says so; calling it security is the single fastest way to lose credibility on everything else you have said.

---

6. "What makes this not a medical device?"

A. It makes no claim to diagnose, treat, prevent, mitigate or monitor a disease. It reports what happened inside the app — rounds completed, first-try answers, days practised, the level the activities are set to — and nothing else. No score is shown to the patient, no threshold means anything clinically, and no output is phrased as a statement about the person. The two modules that could most easily cross that line carry the rule as a comment at the top, and every caregiver screen carries "This is not a diagnostic tool — consult a professional for concerns." What would make it a device is a claim, and we have made none.

PROOF. `js/core/analytics.js:5-6`; `js/core/model.js:12-13`; `content/en.json:640, 696, 743`.

TRAP. Do not say "it could help detect early decline." That single sentence converts the whole project into a regulated product and there is no recovering from it in the room.

---

7. "How is this different from a flashcard app?"

A. Three things. It leads with a recorded, human-sounding voice on every screen and requires no reading at all. It adapts per activity, per patient, on-device, and *shows* that adaptation to the caregiver as prediction plotted against outcome rather than asserting it. And it is a whole-day companion — reminders that arrive on time, a Today screen, a named-photo browser that feeds the recall activity — rather than a drill. The activities are the visible part; the reminders and the caregiver record are what a family keeps it installed for.

PROOF. `js/screens/patient/reminder-card.js:60-74`; `js/screens/caregiver/engagement.js:102-120`; `js/screens/patient/exercises.js:330-351` (caregiver photos become activity rounds).

TRAP. Do not answer with the activity list. Five activities *is* the flashcard app. Answer with the voice, the adaptation you can see, and the day.

---

8. "What is broken in it right now?"

A. Five things, and I would fix them in this order. Reminders cannot fire while the app is closed — that needs an OS-level alarm a web page cannot register without a server. The caregiver side has no automated coverage at all. The voice-status readout overstates: it reports a device as having a voice whenever recordings exist, but eight templated lines still need a live engine, so a phone with no engine gets a green dot and a silent "Tap the gamosa". Route params live outside the URL, so no screen is restorable by reload or hardware-back. And claim discipline is enforced by review rather than by a check.

PROOF. `js/screens/patient/reminder-card.js:6-8`; `tools/demo.html:136-149`; `js/core/voice.js:106-114`; `js/core/router.js:5, 12`.

TRAP. Do not say "nothing." Naming your own weaknesses first is the cheapest credibility in the room, and a limitation a judge discovers is worth ten times the damage of one you volunteered.

---

9. "What did you not build, and why should we believe you could?"

A. Accounts, authentication, encryption, cloud sync, cloud ML, cognitive-condition inference and multi-patient workflows. Every one of them is excluded for a reason that follows from the constraints, not from time: sync and accounts break offline-first and create a data-processor obligation the prototype cannot meet; encryption without a keystore is theatre; condition inference crosses the diagnostic line the project refuses. The reason to believe we could build them is what we did build in the same space — a complete, exportable single-device record, which is exactly the primitive a sync layer would sit on.

PROOF. `README.md:98–106`; `js/screens/caregiver/memories-settings.js:247–272`.

TRAP. Do not say "we ran out of time." It is almost certainly partly true and it is the weakest available answer. The design reason is stronger and it is also the real one.

---

10. "Cold start. Round one, new patient. What does your model do?"

A. Nothing. It predicts and it trains, but it does not steer until it has seen six rounds — before that the rules engine is entirely in charge, which is the behaviour the app had before the learner existed. And the caregiver screen says so in words rather than pretending: "Watching for now — it starts adjusting after 4 more rounds."

PROOF. `js/core/model.js:29–32, 125–132`; `js/screens/caregiver/engagement.js:136–143`; `content/en.json:706`.

TRAP. Do not say "it uses a sensible prior so it's fine from round one." That invites "show me it doesn't lurch," and the honest answer — it does not act at all — is stronger.

---

11. "What stops the model doing something harmful — jumping to the hardest level?"

A. Three bounds, all constants at the top of one file. It only ever considers one level either side of where the auditable rules engine landed. A candidate must be at least 0.03 closer to the target to win, and ties fall to the *easier* level. And it does not act for six rounds. The guardrail is not asserted, it is brute-forced: 1,200 proposals against random histories, zero breaches.

PROOF. `js/core/model.js:120-150`; `tools/model-test.html:105-117`.

TRAP. Do not describe the target as "keeping them at 75 % correct." It is where the model *aims*; with three levels the target is often unreachable exactly, and the harness judges against the best level available, not against 0.75.

---

12. "Show me your data model. What do you store about this person?"

A. Five collections on one device: reminders, photos as downscaled data URLs with names, engagement (per-activity level, streaks, four model weights, and the last 60 rounds), settings, and an activity log. No age, no diagnosis, no clinical field, no free text about the person beyond a photo caption and a reminder note. The model itself sees four numbers: a bias, the level, first-try accuracy over the last 8 rounds, and retry count over the same window.

PROOF. `js/core/store.js:24-33` and `:368-380`; features at `js/core/model.js:75-87`.

TRAP. Do not gesture vaguely at "it's all local." Name the five collections — specificity here is what makes "nothing leaves the device" believable.

---

13. "A 1 GB phone with no TTS voice installed. What happens?"

A. It still speaks. 191 English lines are shipped as recordings and play through an `<audio>` element with no engine involved — the intro, all feedback, the praise, the Exercise 2 and 5 prompts, every fixed line. Eight lines are assembled at runtime from a name, a time or an object — the reminder card, Today's next-reminder sentence, "Tap the gamosa" — and those need a live engine. With none, they are silent and shown as text. That is the gap, and the readout that reports it is the third thing on my fix list.

PROOF. `js/core/voice.js:125–141`; `tools/build-voice.py:68–72`; `js/core/voice.js:106–114`.

TRAP. Do not say "we ship all the audio." Eight lines are not shipped and a judge who opens `assets/audio/en/index.json` will find that in thirty seconds.

---

14. "Accessibility. Tremor, cataracts, hearing loss."

A. Tremor: 64 px minimum touch targets, no swipe, pinch or multi-touch anywhere in normal patient use, and in four of five activities a mis-tap reopens the round rather than ending it — with input held during the pause so a shake cannot double-fire. Vision: 18 px root, 29 px patient body text, 14.5:1 body contrast, a 3 px focus outline. Hearing: every spoken line is also on screen, and nothing in the app waits on or advances only on audio. One honest failure: the "3 of 5" photo counter uses the faintest token and measures 3.03:1, which fails AA for body text.

PROOF. `css/base.css:33, 88–89, 104`; `js/screens/patient/exercises.js:534, 549–558`; `css/patient.css:406`.

TRAP. Do not claim WCAG compliance. You have measured three ratios, not audited a product. Quote the numbers you have and name the one that fails.

---

15. "Scale. 100 patients, one health worker. What breaks first?"

A. Everything, immediately, by design. There is no account and no server, so there is no second patient on one device and no one patient across two devices — the only bridge is a JSON backup moved by hand. A health-worker workflow needs identity, which needs accounts, which needs a server, which breaks the offline constraint the entire app is built on. That is a different product with a different threat model and a different regulatory posture. Building a fake version of it here would have been the dishonest choice.

PROOF. `js/screens/caregiver/memories-settings.js:247-272`; `README.md:98-102`.

TRAP. Do not say "we'd just add a backend." It concedes the architecture is incidental, when in fact offline-first is the whole thesis.

---

16. "Your reminders. Will they wake the patient if the app is closed?"

A. No, and it is written in the source, in the README and in the docs. A web page cannot register an OS-level alarm without a push server, and a push server is a network dependency the app is built to do without. What we built is a 30-second watcher so a reminder arrives while the app is open rather than only when Home is redrawn — and it deliberately does not interrupt an activity in progress. The real fix is a thin native wrapper that owns the alarm and opens the PWA.

PROOF. `js/screens/patient/reminder-card.js:3-8, 16-19, 60-74`.

TRAP. Do not demonstrate a reminder by leaving the app open and calling it "notifications." Say the limit first, then show the watcher working.

---

17. "There's fake data on this device. Are you demoing an invented patient?"

A. Yes, and the app says so on the screen that owns it. A cold install shows an empty state everywhere, which reads as a broken app rather than a new one, so first run seeds a fortnight of plausible history. Caregiver → Memories & Settings opens with a "Sample data" banner explaining that everything shown is an example, and one tap clears it and leaves a blank device. The seeded rounds train the learner exactly as real ones would, so the Engagement screen opens on a model that has genuinely learned from that history rather than a blank one claiming otherwise.

PROOF. `js/app.js:28-32`; `js/screens/caregiver/memories-settings.js:312-327`; `js/core/demo-data.js:74-78`.

TRAP. Do not let them find the banner before you show it.

---

18. "Adding a language. You claim zero code changes — is that true?"

A. True of the application, not of the repository, and I would rather be precise. Drop `content/ne.json` next to the others, add one entry to `app.languages` in each pack, set the `speechLang` fallback chain — and no JavaScript changes, because every string, image path and language tag already comes from the pack. Two things outside the app do change: a line in `tools/build-voice.py` if you want recordings, and an entry in the service worker's `APP_SHELL` so the pack precaches. The two shipped packs are 274 keys each with no divergence, which is what makes the claim checkable.

PROOF. `js/core/content.js:8-17`; `js/app.js:18-20`; `tools/build-voice.py:45-48`; `sw.js:10-11`.

TRAP. Do not say "zero code changes" flatly. The overclaim is small, it is easy to check, and losing a true claim to an unnecessary exaggeration is the worst trade in the room.

---

19. "Why does Assamese speak with a Bengali voice?"

A. Because Assamese neural voices do not exist in the service we build with, and Android phones almost never carry an Assamese system voice. Assamese is written in the Bengali-Assamese script, so a Bengali voice renders the graphemes far closer than any Latin-script voice would. Both layers use the same chain: the build renders Assamese with `bn-IN-TanishaaNeural`, and the live fallback list is `as-IN → as → bn-IN → bn`, so a device that *does* have an Assamese voice gets it. The caregiver screen reports which voice was actually found. The honest caveat: Bengali reading Assamese is approximately, not natively, correct — and the Assamese pack itself is machine-authored and has not been reviewed by a native speaker.

PROOF. `tools/build-voice.py:41-48`; `content/as.json:8-13`; `js/core/voice.js:34-38`; `README.md:111-112`.

TRAP. Do not present the Assamese pack as production-ready. If there is a Assamese speaker on the panel, that claim ends badly in one sentence.

---

20. "How do I know the JavaScript you ship is the model you validated?"

A. A golden trace. `tools/model-harness.py --golden` runs a fixed 20-round outcome pattern through the Python mirror and prints the resulting levels, weights and prediction. `tools/model-test.html` runs the identical pattern through the real `js/core/model.js` in the browser and asserts all three match — weights to nine decimal places. And the mirror reads every constant straight out of the JavaScript source by regex, so it cannot silently drift from the code it is checking. I re-ran the generator against current source before this review: the three constants pinned in the test file match exactly.

PROOF. `tools/model-harness.py:32-53, 231-251`; `tools/model-test.html:36-38, 88-99`.

TRAP. Do not say "it's the same algorithm." Say "same constants, read from the source, and a golden trace that pins them to nine decimals."

HONEST WEAKNESSES — the five I would fix first

Ranked. Not softened.

1. **Reminders cannot fire while the app is closed.** This is the largest gap between what the product promises and what it does. A web page cannot register an OS alarm without a push server. Needs a thin native wrapper. — `js/screens/patient/reminder-card.js:6-8`

2. **The caregiver side has no automated coverage at all.** Adding a reminder, adding a photo, changing language, export and restore are hand-tested only. `tools/demo.html` renders those screens but asserts nothing about their mutations. This is where a live demo breaks. — `tools/demo.html:136-149`

3. **The voice-status readout overstates what the device can say.** It reports a voice as available whenever recordings exist, but eight templated lines still need a live engine — so a phone with no TTS gets a green dot and a silent "Tap the gamosa." — `js/core/voice.js:106-114`

4. **Route params live outside the URL.** No screen is addressable or restorable; reload and Android hardware-back replay whatever params were set last. Invisible on the patient rails today, but it is a latent bug in a demo where anyone might press back. — `js/core/router.js:5, 12`

5. **Claim discipline is enforced by review, not by a check.** The entire user-visible surface is two JSON files; a twenty-line lint over them for the forbidden vocabulary would make the project's most

> important guarantee structural instead of cultural. It does not exist. — `content/en.json`, `content/as.json`

Runners-up worth knowing before someone else finds them: the localStorage mirror swallows quota failures silently (`js/core/store.js:93-99`); the manifest's maskable icon is not in the precache (`manifest.json:12` vs `sw.js:38-43`); `clearAll()` leaves a freshly wiped device reporting "Last active: Just now" (`js/core/store.js:484-486`); speech input is wired into Exercise 1 only (`js/screens/patient/exercises.js:403`); and the photo counter's contrast fails WCAG AA (`css/patient.css:406`).

Vocabulary: engagement, activity performance, session history, practice activities. The activities are exercises or activities, never games. No claim that this app measures, detects, diagnoses or improves cognition, because it does none of those things.